

Raport laboratoryjny

Gra Breakthrough — Minimax i cięcia alfa-beta

Zakres. Implementacja gry planszowej Breakthrough z ograniczonym przeszukiwaniem Minimax, heurystykami, cięciami alfa-beta oraz trzema typami agentów w CLI (`minimax`, `random`, `epsilon-greedy`).

Dawid Błaszczyk

Spis treści

1. [Wstęp teoretyczny](#)
 2. [Sformułowanie problemu](#)
 3. [Idea rozwiązania](#) — w tym [odniesienie do punktacji listy](#)
 4. [Optymalizacje](#)
 5. [Opis heurystyk](#)
 6. [Testy automatyczne](#)
 7. [Wnioski z partii AI kontra AI](#)
 8. [Biblioteki](#)
 9. [Uruchomienie](#)
 10. [Przykładowa rozegrana partia](#)
-

1. Wstęp teoretyczny

1.1. Algorytm Minimax

Minimax to strategia stosowana w grach takich jak szachy czy kółko i krzyżyk. Opiera się na założeniu, że obaj gracze grają w sposób optymalny, a ich cele są całkowicie sprzeczne.

- **Gracz maksymalizujący** — dąży do ruchów z jak najwyższą oceną.
- **Gracz minimalizujący** — dąży do ruchów, które zminimalizują wynik przeciwnika.

Algorytm analizuje grę w formie drzewa decyzyjnego: sprawdza możliwe ruchy jednego gracza, potem odpowiedzi drugiego. Ze względu na czas zwykle nie dochodzi do liści „prawdziwego” końca gry — zatrzymuje się na ustalonej głębokości i ocenia stan **heurystyką**.

1.2. Suma zerowa i suma niezerowa

Działanie Minimaxu opiera się głównie na koncepcji sumy zerowej.

- **Suma zerowa** — zysk jednego gracza jest stratą drugiego (np. +1 i -1 dają w sumie 0). Interesy są sprzeczne, nie ma kompromisu.
- **Suma niezerowa** — obie strony mogą naraz zyskać lub stracić; Minimax wtedy bywa zawodny, bo zakłada zawsze „walkę”, a nie współpracę.

1.3. Cięcia alfa-beta

Technika przyspieszająca Minimax przez pomijanie gałęzi drzewa, które **nie zmieniają** ostatecznej decyzji u korzenia.

- **Alfa (α)** — najlepsza wartość, jaką gracz maksymalizujący ma już zagwarantowaną na ścieżce.
- **Beta (β)** — najlepsza (najniższa) wartość zabezpieczona przez gracza minimalizującego.

Gdy dalsza analiza ścieżki nie poprawi wyniku, następuje **odcięcie** — oszczędność czasu i węzłów przy tej samej decyzji co w zwykłym Minimaxie (przy optymalnym porządku przeglądania).

2. Sformułowanie problemu

2.1. Stan gry

Stan opisujemy jako:

- **Plansza** — prostokątna siatka; domyślnie 8×8 , rozmiar ustawia się w CLI (`--rows` , `--cols`). Pola: pion gracza pierwszego (B), drugiego (W), puste (`_`), pole startu ostatniego ruchu (`o`).
- **Tura** — który gracz ma ruch.
- **Start** — z `_default_start_position()` : głębokość obozu `min(2, rows // 2)` rzędów od każdej krawędzi (dla klasycznego 8×8 to dwa rzędy B u góry i dwa W u dołu).
- **Koniec gry**

- *Dotarcie do mety* — pion w ostatnim rzędzie po stronie przeciwnika.
- *Blokada* — przeciwnik nie ma żadnego legalnego ruchu (w tym brak pionów).

2.2. Ruchy (przejsćia)

- **Prosto** — o jedno pole do przodu, tylko na puste.
- **Po skosie** — na puste albo na przeciwnika (**bicie**).
- **Bez bicia wprost** — bicie wyłącznie z ruchu po skosie.

2.3. Drzewo decyzyjne i Minimax

- **Węzły** — stany gry.
- **Krawędzie** — legalne ruchy.
- **Minimax** — wybór ruchu przy założeniu optymalnej gry przeciwnika; przy limicie głębokości d liście oceniane są heurystyką.

3. Idea rozwiązania

Moduły współpracują ze sobą w następujący sposób:

Moduł	Rola
Silnik (Board)	Legalne ruchy, wykonanie ruchu, wygrana.
Minimax + alfa-beta	Przeszukiwanie z limitem głębokości; licznik węzłów i czas na stderr.
Heurystyki	Co najmniej trzy strategie oceny na gracza (w projekcie jest ich sześć).
Agenci	Minimax, losowy, epsilon-greedy; osobne parametry dla B i W; opcjonalna adaptacja heurystyki.

Przykład: Przy głębokości $d = 3$ ruch prosty może odsłonić bicie, a ruch po skosie przybliżyć do mety — algorytm wybierze wariant z lepszą wartością minimaxową według heurystyki.

3.1. Odniesienie kodu do punktacji z listy

Ponizj: który fragment kodu odpowiada któremu punktowi z listy ćwiczeniowej.

Zadanie 1 — stan gry i legalne ruchy (10 pkt)

Plansza to `Board.grid`. Start z `_default_start_position()` lub wczytanie `from_lines`. Ruchy zbiera `get_legal_moves`; pole o traktowane jak puste (`_is_empty`).

```
# engine/board.py

class Board:
    def __init__(
        self,
        grid: Sequence[Sequence[str]] | None = None,
        rows: int | None = None,
        cols: int | None = None,
    ) -> None:
        ...

    def get_legal_moves(self, player: Player) -> List[Move]:
        ...
```

Zadanie 2 — zbiór heurystyk (20 pkt)

Szesć funkcji oceny zarejestrowanych w `HEURISTICS`.

```
# ai/heuristics.py

HEURISTICS: Dict[str, Heuristic] = {
    "material": material_heuristic,
    "advancement": advancement_heuristic,
    "mobility": mobility_heuristic,
    "goal_pressure": goal_pressure_heuristic,
    "center_control": center_control_heuristic,
    "threatened_pieces": threatened_pieces_heuristic,
}
```

Zadanie 3 — Minimax i wersja podstawowa programu (30 pkt)

- `_minimax` — wartość pozycji w drzewie.
- `choose_best_move` — wybór najlepszego ruchu u korzenia.
- Domyslnie obaj agenci `minimax` — wersja podstawowa z listy.

```
# ai/minimax.py

def _minimax(board, depth, current_player, root_player, heuristic,
alpha, beta, use_alpha_beta, counters) -> float:
    counters["visited_nodes"] += 1
    terminal_score = _evaluate_terminal(board, root_player,
current_player)
    if terminal_score is not None:
        return terminal_score
    if depth == 0:
        return heuristic(board, root_player)
    # legalne ruchy, max/min, copy_board, apply_move, rekurencja

def choose_best_move(board, player, depth, heuristic_name,
use_alpha_beta=True) -> SearchResult:
    # dla każdego ruchu z korzenia: _minimax(..., depth - 1, ...); wybór
    najwyższego wyniku
```

```
# main.py – jedna iteracja pętli gry

    move, visited_nodes, elapsed_seconds = choose_move_for_agent(
        board=board,
        player=current_player,
        agent_type=agent_type,
        depth=depth,
        heuristic_name=heuristic_name,
        use_alpha_beta=not args.no_alpha_beta,
        epsilon=epsilon,
    )
    board.apply_move(move, current_player)
    current_player = player_2 if current_player.symbol == "B" else
player_1
```

Zadanie 4 — alfa-beta w wersji podstawowej (40 pkt)

W `_minimax`, przy włączonym `use_alpha_beta`, pętlę można przerwać, gdy `beta <= alpha`.

```
# ai/minimax.py
```

```
    if use_alpha_beta and beta <= alpha:  
        break
```

Zadanie 5 — wersja rozszerzona (20 pkt)

- Osobny tryb gry dla B i W: `--agent-p1`, `--agent-p2`.

```
# main.py
```

```
    parser.add_argument("--agent-p1", choices=["minimax", "random",  
"epsilon-greedy"], default="minimax")  
    parser.add_argument("--agent-p2", choices=["minimax", "random",  
"epsilon-greedy"], default="minimax")  
    parser.add_argument("--adaptive-strategy", action="store_true",  
                        help="Enable dynamic heuristic switching based  
on board state.")
```

```
# ai/agent_logic.py

def choose_adaptive_heuristic(board, player, fallback_heuristic):
    if _distance_to_goal(board, player) <= 2:
        return "advancement"
    if material_heuristic(board, player) < 0:
        return "material"
    if fallback_heuristic == "advancement":
        return "mobility"
    return fallback_heuristic

def choose_move_for_agent(board, player, agent_type, depth,
    heuristic_name, use_alpha_beta, epsilon):
    legal_moves = board.get_legal_moves(player)
    if not legal_moves:
        return None, 1, 0.0
    if agent_type == "random":
        return random.choice(legal_moves), 1, 0.0
    if agent_type == "epsilon-greedy" and random.random() < epsilon:
        return random.choice(legal_moves), 1, 0.0
    search = choose_best_move(
        board=board, player=player, depth=depth,
        heuristic_name=heuristic_name, use_alpha_beta=use_alpha_beta,
    )
    return search.best_move, search.visited_nodes,
    search.elapsed_seconds
```

4. Optymalizacje

- **Alfa-beta** — nie trzeba przeglądać każdej gałęzi jak w „gołym” minimaksie; wynik decyzji ten sam przy mniejszej liczbie węzłów.
- **Limit głębokości** — przy `depth == 0` zamiast dalszego rozgałęziania jest od razu heurystyka.

W `_minimax`, po aktualizacji `alpha` / `beta`, przy włączonym cięciu:

```
# ai/minimax.py

    if is_maximizing:
        best_value = max(best_value, evaluation)
        alpha = max(alpha, best_value)
    else:
        best_value = min(best_value, evaluation)
        beta = min(beta, best_value)

    if use_alpha_beta and beta <= alpha:
        # [Punkt 4] Alfa-beta: odcinamy gałąź, która nie wpłynie na
decyzję.
        break
```

Przy wyborze ruchu od korzenia `alpha` i `beta` startują od $-\infty$ i $+\infty$ (`choose_best_move`).

Lisć przy braku głębokości:

```
# ai/minimax.py

    if depth == 0:
        return heuristic(board, root_player)
```

Konfiguracja z `main.py` : głębokość (`--depth` , `--depth-p1` , `--depth-p2`) oraz wyłączenie cięć (`--no-alpha-beta`).


```

# main.py

    parser.add_argument(
        "--depth",
        type=int,
        default=3,
        help="Default maximum search depth for Minimax agents (default:
3).",
    )
    parser.add_argument(
        "--depth-p1",
        type=int,
        default=None,
        help="Maximum search depth for player 1 agent.",
    )
    parser.add_argument(
        "--depth-p2",
        type=int,
        default=None,
        help="Maximum search depth for player 2 agent.",
    )
    parser.add_argument(
        "--no-alpha-beta",
        action="store_true",
        help="Disable alpha-beta pruning.",
    )

    move, visited_nodes, elapsed_seconds = choose_move_for_agent(
        board=board,
        player=current_player,
        agent_type=agent_type,
        depth=depth,
        heuristic_name=heuristic_name,
        use_alpha_beta=not args.no_alpha_beta,
        epsilon=epsilon,
    )

```

5. Opis heurystyk

Każda funkcja zwraca **jedną liczbę** dla wybranego gracza — im wyżej, tym pozycja lepsza dla niego.

Nazwa (CLI)	Idea
<code>material</code>	Różnica liczby pionów (ja – przeciwnik).
<code>advancement</code>	Jak daleko piony weszły w stronę mety.
<code>mobility</code>	Różnica liczby legalnych ruchów.
<code>goal_pressure</code>	Kto ma bliżej do mety (najszybszy pion).
<code>center_control</code>	Bonus za środek planszy (kolumny i pas wierszy).
<code>threatened_pieces</code>	Bilans pionów zagrożonych biciem z przodu.

Heurystyki nie widzą całej partii do końca — przy małej głębokości możliwe są błędy w długiej taktyce.

6. Testy automatyczne

Plik	Co sprawdza
<code>test_equivalence_score</code>	Ten sam wynik u korzenia z Minimaxem i z alfa–beta.
<code>test_equivalence_nodes</code>	Nie więcej węzłów z cięciami; przy $d = 3$ gdzieś ściśle odcięcie.
<code>test_puzzles</code>	Pozycje taktyczne (m.in. wejście na metę, „musisz zbic”).
<code>test_brute_force</code>	Zgodność wyniku z referencyjnym „naiwnym” minimaksem.
<code>test_argmax</code>	Wybrany ruch ma najwyższą wartość i zgadza się z raportowanym wynikiem.

Uruchomienie z katalogu `task-2/` :

```
python3 tests/run_all.py
```

Przykładowy wynik:

```
Minimax and alpha-beta return the same root score
-----

Alpha-beta visits no more nodes than plain Minimax
-----

Alpha-beta strictly prunes at depth 3 on a representative position
-----

Tactical puzzles: mate-in-1 and capture-or-lose
-----

Production score equals independent brute-force Minimax
-----

Reported best move is an argmax over legal children
-----

Ran 906 checks in 18.989s: 906 passed, 0 failed
```

7. Wnioski z partii AI kontra AI

- Przy **tej samej** głębokości i heurystyce wynik zależy od `--seed` i od tego, co heurystyka „widzi” w horyzoncie d .

Obserwacje z prób. Przy `advancement`, głębokość 2, `seed=1` często wygrywał gracz drugi po około 50 pół-ruchach. Przy `material` vs `advancement`, $d = 3$, `seed=2`, partia była dłuższa (rząd 90 pół-ruchów), z podobnym wynikiem w próbie. Przy różnej głębokości (2 vs 3, `seed=1`) głębsze przeszukiwanie **nie gwarantowało** wygranej tej strony.

Wniosek praktyczny. Heurystyka mówi *jak* program punktuje pozycję, a głębokość *jak* daleko patrzy w przód — oba ustawienia trzeba dobierać łącznie do dostępnego czasu; sama

większa głębokość bez sensownej funkcji oceny niewiele da, bo na liściach i tak obowiązuje przybliżenie.

8. Biblioteki

Program używa wyłącznie **biblioteki standardowej** Pythona:

Moduł	Krótko
<code>argparse</code>	Opcje z linii poleceń (głębokość, heurystyki, agenci, <code>--no-alpha-beta</code>).
<code>dataclasses</code>	<code>Move</code> , <code>Player</code> , <code>SearchResult</code> .
<code>pathlib</code> , <code>random</code> , <code>sys</code>	Log, losowość, stderr z metrykami.
<code>time.perf_counter</code>	Pomiar czasu Minimaxa.
<code>math.inf</code>	Start α, β i granice w Minimaxie.

9. Uruchomienie

Z katalogu `task-2/src/`:

```
python3 main.py --depth 4 --heuristic-p1 advancement --heuristic-p2
mobility
python3 main.py --agent-p1 minimax --agent-p2 epsilon-greedy --depth-p1
4 --depth-p2 2 --adaptive-strategy
python3 main.py --no-alpha-beta
python3 main.py --board-from-stdin --depth 3
python3 main.py --rows 6 --cols 10 --depth 2
```

Wersja podstawowa (Minimax vs optymalny przeciwnik, ta sama heurystyka)

Rozegranie całej partii tak, by **gracz 1 (B) i gracz 2 (W) byli agentami minimax**, używali **tej samej** heurystyki i **tej samej** głębokości — wtedy model „przeciwnik gra optymalnie według tej samej funkcji oceny” jest spójny z faktycznym ruchem W.

Z katalogu `task-2/src/`:

```
python3 main.py --depth 3
```

To odpowiada jawnemu zapisowi (domyślne agenty to i tak `minimax`, domyślna heurystyka to `advancement` dla obu stron):

```
python3 main.py --agent-p1 minimax --agent-p2 minimax --heuristic-p1 advancement --heuristic-p2 advancement --depth 3
```

Wersja rozszerzona (dwaj odrębni agenci, strategia może się różnić)

Różne `--agent-p1` / `--agent-p2` i/lub różne głębokości, heurystyki lub `--adaptive-strategy`.

Przykłady z `task-2/src/`:

```
python3 main.py --agent-p1 minimax --agent-p2 epsilon-greedy --depth-p1 4 --depth-p2 2 --epsilon-p2 0.2
python3 main.py --agent-p1 minimax --agent-p2 random --depth-p1 3
python3 main.py --agent-p1 minimax --agent-p2 minimax --heuristic-p1 advancement --heuristic-p2 material --depth-p1 4 --depth-p2 3
python3 main.py --agent-p1 minimax --agent-p2 minimax --depth 3 --adaptive-strategy
```

Pełna lista flag: `python main.py --help`.

10. Przykładowa rozegrana partia

Ponizżej: **pięć** migawkowych plansz wyciętych z pliku logu (`task-2/logs/...`) — widac, jak przesuwają się piony B (gracz 1) i W (gracz 2).

```
python3 main.py --rows 6 --cols 6 --depth 2 --heuristic-p1 advancement -  
-heuristic-p2 advancement --agent-p1 minimax --agent-p2 minimax --seed 0  
--max-rounds 120 --log-file readme_maps.txt
```

Mapa 1 — plansza startowa

```
B B B B B B  
B B B B B B  
- - - - -  
- - - - -  
W W W W W W  
W W W W W W
```

Mapa 2 — po rundzie 4

```
_ B B B B B  
B B B B B B  
B _ _ _ _  
W _ W _ _ _  
_ o W W W W  
W W W W W W
```

Mapa 3 — po rundzie 8

```
_ _ B B B B  
B B B B B B  
B _ B _ _ _  
W _ W _ W W  
_ _ W _ o W  
W W W W W W
```

Mapa 4 — po rundzie 12

```
_ _ B B B B
_ B _ B B B
B W B _ _ _
_ _ o _ W W
_ _ W _ _ W
W W W W W W
```

Mapa 5 — po rundzie 16

```
W _ _ B B B
o B B B B B
_ _ B _ _ _
B _ _ _ W W
_ _ W _ _ W
W W W W W W
```

W ostatniej planszy **W** stoi w **pierwszym rzędzie**, co oznacza, że partia kończy się wygraną W.

```
--- GAME END ---
rounds=16
winner=Player 2
visited_nodes=4744
time_s=0.230197
```